Basi di dati — Tecnologie

Università di Verona
Imbriani Paolo - VR500437
Professor Sara Migliorini

March 3, 2026

Contents

1 SQL **3**
1.1 Istruzioni 3
1.2 SELECT 4
1.3 FROM 4
1.4 WHERE 5
1.4.1 LIKE 5
1.4.2 SIMILAR TO 5
1.4.3 IN 6
1.4.4 IS NULL 6

1 SQL

SQL è il linguaggio di riferimento per le basi di dati relazionali. Originariamente SQL era l'acronimo di "Structured Query Language", ma oggi è semplicemente SQL. SQL ha varie funzionalità:

- **Data Definition Language (DDL)**: consente di definire la struttura del database, come la creazione di tabelle, l'aggiunta di colonne e la definizione di chiavi primarie.
- **Data Manipulation Language (DML)**: consente di manipolare i dati all'interno del database, come l'inserimento, l'aggiornamento e la cancellazione di record.
- **Data Query Language (DQL)**: consente di interrogare il database per recuperare informazioni specifiche.

Ne esistono inoltre varie versioni. In laboratorio useremo SQL-2 poiché sappiamo che è supportato da tutti i DBMS.

1.1 Istruzioni

La sintassi di base è:

```
SELECT [ DISTINCT ]
[ * | expression [[ AS] output_name ] [, ...] ]
[ FROM from_item [, ...] ]
[ WHERE condition ]
[ GROUP BY grouping_element [, ...] ]
[ HAVING condition [, ...] ]
[ { UNION | INTERSECT | EXCEPT } [ DISTINCT ] other_select ]
[ ORDER BY expression [ ASC | DESC | USING operator ]]
```

- expression è un' espressione che determina un attributo.
- from_item è un' espressione che determina una sorgente per gli attributi.
- condition è un' espressione booleana per selezionare i valori degli attributi.
- grouping_element è un'espressione per poter eseguire operazioni su più valori di un attributo e considerare il risultato
- SELECT, FROM e WHERE sono chiamate clausole.
- Estrae dal prodotto cartesiano delle tabelle elencate nella clausola FROM, le righe che soddisfano la condizione specificata nella clausola WHERE.

– Il risultato dell'interrogazione è una tabella con una riga per ogni riga prodotta dalla clausola FROM ed eventualmente filtrata dalla clausola WHERE (se esiste), le cui colonne dipendono dalla lista degli attributi nella clausola SELECT.

1.2 SELECT

```
SELECT [ DISTINCT ]
[{* | expression [[ AS] output_name ]} [, ...]]
```

- è un'abbreviazione per indicare tutti gli attributi delle tabelle.
- expression è un'espressione che coinvolge gli attributi della tabella (può anche essere semplicemente il nome di un attributo).
- output_name è il nome assegnato all'attributo che conterrà il risultato della valutazione dell'espressione expression nella relazione risultato.
- DISTINCT: se presente richiede l'eliminazione delle tuple duplicate.

1.3 FROM

```
FROM from_item [, ...]
```

1. `table_name [[AS] alias [(column_alias [, ...])]`
   - Se sono presenti due o più tabelle, si esegue il prodotto cartesiano tra tutte le tabelle. Se ci sono attributi con lo stesso nome su tabelle diverse, nella SELECT questi attributi sono identificati anteponendo il nome della tabella e un ".": nomeTabella.nomeAttributo
2. `(other_select) [AS] nomeRisultato [(column_alias [,...])]`
   - è una SELECT innestata. Il risultato della SELECT interna è lo schema con nome nomeRisultato su cui fare la SELECT corrente.
3. `from_item [NATURAL] join_type from_item [ON condition]`
   - è la clausola JOIN.
   - Metodo più sofisticato che permette di selezionare un sottoinsieme del prodotto cartesiano di due o più tabelle. join_type specifica il tipo di join. I tipi principali sono riassunti nelle slide seguenti.

1.4 WHERE

```
WHERE condition
```

La clausola WHERE ammette come argomento un'espressione booleana costruita combinando predicati semplici con gli operatori AND, OR e NOT

- AND: seleziona le righe in cui tutti i predicati sono veri
- OR: seleziona le righe in cui almeno un predicato è vero
- NOT: seleziona le righe in cui il predicato è falso

Ciascun predicato semplice usa gli operatori $=, <>, <, >, <=$ e $>=$ p er confrontare un'espressione costruita a partire dal valore di un attributo con un valore costante, oppure un'altra espressione. Per esprimere la precedenza degli operatori AND e OR si fa uso della parentesi.

1.4.1 LIKE

Per il confronto tra stringhe, SQL mette a disposizione anche l'operatore LIKE che consente verificare se una stringa rispetta un certo "pattern" (oppure ILIKE per case insentive).
Due caratteri speciali per il confronto "_" e "%":

- "_": confronto con un carattere arbitrario
- "%": confronto con una stringa di lunghezza arbitraria (anche 0) di caratteri arbitrari

'$_o\%i$' = stringa che ha una o in seconda posizione, seguita da una sequenza qualsiasi di caratteri (anche vuota) e che termina con una i.

1.4.2 SIMILAR TO

L'operatore SIMILAR TO è un LIKE più espressivo che accetta, come pattern, un sottoinsieme delle espressioni regolari POSIX (versione SQL).
Esempi di componenti di espressioni regolari:

- _: confronto con un carattere arbitrario
- %: 0 o più caratteri arbitrari
- : ripetizione dell'elemento precedente (carattere o gruppo) 0 o più volte
- +: ripetizione dell'elemento precedente (carattere o gruppo) 1 o più volte
- $\{n, m\}$: ripetizione del precedente match almeno n e non più di m volte.
- [...]: elenco di caratteri ammissibili

1.4.3 IN

Nella clausola WHERE può apparire l'operatore [NOT] IN per testare l'appartenenza di un valore ad un insieme.

```
WHERE attributo [NOT] IN ( valore [, ...] );
```

1.4.4 IS NULL

Per verificare se un attributo ha un valore nullo o meno si utilizza il predicato IS NULL.

- Attributo IS NULL: ritorna true se l'attributo ha valore nullo
- IS NOT NULL: è la negazione, ritorna true se l'attributo ha valore diverso da nullo

```
WHERE attributo IS [ NOT ] NULL;
```

SQL-2 implementa la logica a 3 valori:

- Un predicato semplice restituisce il valore NULL quando uno qualsiasi dei termini del predicato ha valore nullo.
- Il predicato IS NULL è una eccezione, restituendo sempre il valore vero oppure false, mai il valore NULL.